Consensus-free task ownership for UAV swarms under partition:

deterministic simulation and multi-vehicle ArduPilot software-in-the-loop validation

Charalampos Nadiotis

eudis-swarm project, EUDIS Defence Hackathon 2026, Challenge 2

October 2026

Abstract

A swarm that loses radio contact cannot tell a silent aircraft from a dead one. We present a coordination scheme in which every UAV acts only on evidence it has actually received: task ownership is a replicated set of immutable, leased claims; a peer is declared failed only by a quorum of votes carried over modelled links; and work held by an aircraft that has gone quiet is recovered when its lease expires, without any agreement. In a deterministic simulator we show that this recovery keeps working exactly where consensus cannot: with two aircraft a loss is undeclarable, yet distributed ownership completes the mission while a central coordinator never finishes, and across 440 jammed runs central control failed 68 missions and distributed control none, with identical failure detection in both. We then drive the same, unchanged coordination logic with real autopilot firmware: a software-in-the-loop harness flies four ArduCopter (and mixed ArduCopter/ArduRover) instances in lock-step with the simulator, commands a physically failed vehicle into a safe hold, and records a per-step evidence log. In repeated runs every mission completed, the failed vehicle stopped, its task was recovered by the survivors and no task ever had two owners. Finally we tune the vehicles' position controllers by measurement, cutting the mission time from 14.50 s to 11.92 s and the recovery after a failure from 7.17 s to 4.92 s. All results are firmware on simulated airframes; none is a flight result.

1 Introduction

Contested airspace is not hypothetical. The European Union Aviation Safety Agency lists the Nicosia flight information region among those affected by GNSS jamming and spoofing [1], and a peer-reviewed flight experiment in that region recorded a receiver losing positioning for most of the time aloft [2]. A swarm operating there must expect its own links to fail in the same way: partially, intermittently, and with no message announcing it.

The difficulty is epistemic. When a UAV stops hearing a peer, the peer may be destroyed, jammed, behind terrain, or briefly out of range and still flying. Treating silence as failure discards that peer's work and reassigns it, which duplicates effort whenever the peer was alive. Waiting for certainty is worse: deciding that a process has failed is a consensus problem, and consensus is impossible to guarantee under asynchrony with even one faulty participant [3]. Practical systems escape through failure detectors that may be wrong [4], but a central reassignment that waits for such a detector inherits its blind spots.

This paper takes the opposite route. Ownership is held through leases [5] that each UAV's own replica ages on its own clock; a task whose owner has gone quiet becomes claimable again when the local copy of its lease expires, whether or not anyone has agreed the owner is gone. Disagreement during a partition is permitted; the guarantee is that once connectivity is stably restored every replica converges on one owner per task and the losing claimant stands down on its own.

Contributions.

1. A receiver-local coordination design (Section 3) with three strictly separated layers, leased immutable claims, quorum-only failure declaration and deterministic store-and-forward evidence flooding, enforced by tests rather than convention.

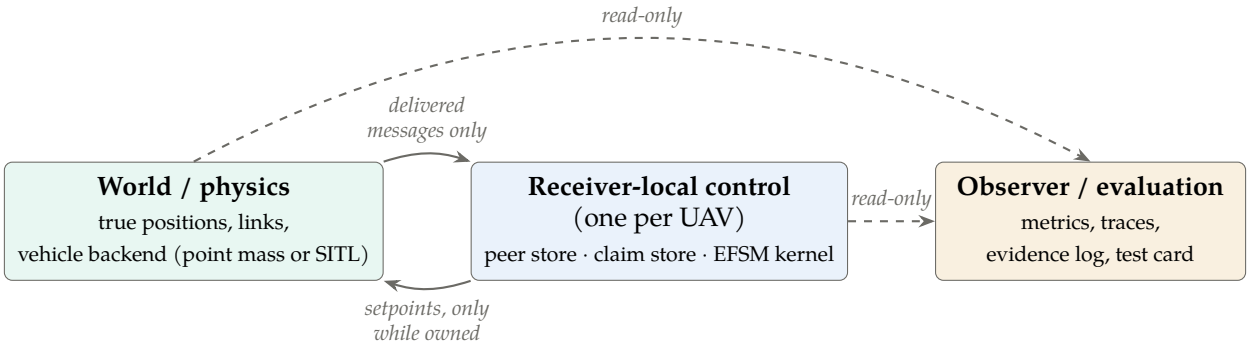


Figure 1: The three layers. A UAV's control state is fed only by messages a modelled link delivered; the observer reads every replica at once (a view no UAV has) and never feeds back.

2. Deterministic evidence (Section 4) that consensus-free recovery survives the conditions that make consensus impossible, in a minimal two-aircraft case and in 440-run populations.
3. A firmware-in-the-loop harness (Section 5) that drives the unchanged coordination logic with several ArduPilot [6] instances: anchored lock-step with measured drift, a one-shot safe-hold fail-stop, a shared mission frame, a heterogeneous copter/rover swarm, an evidence log and a mechanical pass/fail test card.
4. Repeated SITL missions meeting that card (Section 6), and a measured tune of the vehicles' position control that shortens missions and recovery (Section 7).

2 Related work

Partition-tolerant shared state for robot swarms has been approached through distributed ledgers: SwarmDAG [7] maintains a partition-tolerant DAG under extended virtual synchrony. Our goal is narrower and lighter: one owner per task after healing, with no global log. Communication-aware allocation [8] and coverage under communication constraints [9] model links as a planning constraint; we instead make every decision depend only on what a link actually delivered. Distributed 3-D search with a varying number of agents [10] addresses joining and leaving agents in planning; supervisory control theory [11] motivates our formulation of per-UAV behaviour as composed extended finite-state machines. Heterogeneous multi-UAV benchmarks such as CARIC [12] evaluate planning quality; our evaluation is instead about ownership correctness under loss, and about carrying the same logic onto real autopilot firmware.

3 Design

Layers. Figure 1 shows the load-bearing decision. The *world* holds true positions and the communication graph; the *receiver-local control* of each UAV holds its own pose, an immutable task catalogue, a peer-state store, a claim store and a kernel of composed extended finite-state machines (contact, peer availability, per-task ownership, coordination mode); the *observer* computes metrics by reading every replica at once. No topology fact is ever copied into a UAV's state: a receiver knows only what it received, and when.

Claims and leases. Before binding work a UAV publishes an immutable claim $(task, owner, epoch)$. Each replica classifies a stored claim by its *receiver-local* age a: fresh while $a \le \tau_f$, stale while $\tau_f < a \le \tau_\ell$, expired once $a > \tau_\ell$ (by default $\tau_\ell = 2.5\,\mathrm{s}$ and $\tau_f = \tau_\ell/2$). Stale is not unclaimed: a replica may create a competing claim only once its own copy has expired. Epochs are compared only within one owner's stream; reconciliation supersedes older publications per owner and then picks the lowest validated owner identifier, so every replica that has seen the same evidence picks

Table 1: The quorum floor (`demo-quorum-floor` / `baseline-quorum-floor`).

aircraft	control	declared	never detected	tasks	mission
2	central	0	1	2/3	**never finishes**
2	distributed	0	1	3/3	completed at 10.75 s
3	central	1	0	3/3	completed
3	distributed	1	0	3/3	completed

the same winner. A UAV moves toward, renews or completes a task only while its own replica says it owns it.

Failure is a declaration, not an observation. Silence, staleness and unknown state never imply failure. Declaring peer j failed requires votes from a quorum of $q(N) = \max(2, \lfloor (N-1)/2 \rfloor + 1)$ observers, carried over modelled links. One observer can therefore never declare a peer dead, and a declaration never stops a vehicle: it is evidence, and first-hand contact from a "dead" peer retracts it.

Evidence transport. Heartbeats are one hop and never forwarded. Votes, declarations, claims, releases and completion evidence flood the current connected component by deterministic store-and-forward: stable message and neighbour order, per-receiver duplicate suppression, no TTL and no random backoff.

Determinism. The simulator has no wall-clock time, no unseeded randomness and no order-dependent iteration; the same command line produces byte-identical output on any machine. This is what lets exact numbers be asserted by tests and quoted below.

4 Deterministic evaluation

All numbers in this section are evaluator-computed and reproducible from the commands in Appendix A; they are taken from the project's measured-results record.

The quorum floor. With two aircraft the survivor is the only observer, so $q(2) = 2$ makes the loss undeclarable. Both control modes are then equally blind; what differs is whether recovery needed the declaration (Table 1). Central reassignment waits for a declaration that cannot arrive and holds a task for an aircraft that has stopped; distributed ownership recovers through lease expiry. A third aircraft restores the quorum and both modes recover.

Populations under jamming. Across 440 runs (four aircraft, eleven communication ranges, twenty seeds each, a circular jammer), central control failed 68 of 220 missions and distributed control none (Figure 2). A failed central mission completes 19 of 20 tasks and then burns its 300 s budget holding the last task for an aircraft it cannot reach. Failure *detection* was identical in both modes at every range (for example 18/20 seeds undetected at range 50 in both): the two control models have the same awareness and opposite outcomes.

Opposite failure signatures. With a partition that heals, replicated ownership strands no work and pays with brief double-booking (0.286 task-seconds on average over 240 runs), while central control strands work it handed to aircraft it could not reach (4.786 task-seconds on average) and showed no measured double-booking in that sweep. Neither is better; they are wrong in different directions, and an operator "hold" policy trades 64 % of the double-booking for 59 % more mission time.

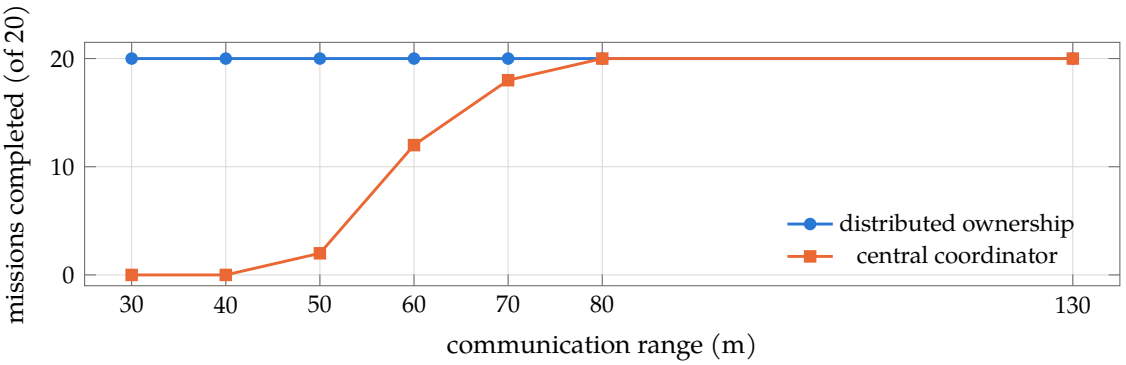


Figure 2: Missions completed versus communication range under a jammer (20 seeds per point; ranges 90–120 omitted, all 20/20 in both modes). Source: measured-results record, section 5.

5 Firmware in the loop

The simulator's motion enters through one interface, `move_toward(agent, target, elapsed)`. Its default implementation is a point mass; the SITL backend implements it with a real autopilot. Nothing in the coordination layer changes, and nothing in the simulator imports the backend (asserted by test).

Anchored lock-step. Logical time advances in fixed steps; each SITL instance free-runs on its own clock. Stepping several instances one after another lets every clock run on while the others are waited on. The first version waited until "last observed time plus the step", which rounds every step up to the next telemetry report: on real hardware it ran the vehicle clock 1.6× ahead of logical time. Each vehicle is now anchored once, all in one pass after draining the telemetry backlog, and a step waits only until $t_{\text{anchor}} + \sum \text{elapsed}$ on that vehicle's own clock. A report is accepted only if it is strictly newer than the last one, and the residual lead (drift) is recorded every step. Anchoring cannot slow a fast instance; it makes the lead visible instead.

Fail-stop. When the simulator's fault schedule makes an integrated agent physically non-responsive, the backend sends that vehicle exactly one safe-hold mode change (ArduCopter `BRAKE` or `LOITER`, ArduRover `HOLD`) and records whether the autopilot confirmed it. It reads only physical truth, never the swarm's belief, so a wrongly declared UAV keeps flying.

Shared frame and deconfliction. All vehicles report in one local frame about a shared origin; the launcher starts instance i at its agent's simulated start position. Copters take off to, and every setpoint holds, their own altitude band $(20\,\text{m} + 5k)$; rovers stay on the ground. Separation is evidenced, not asserted: the log carries each vehicle's observed altitude and the report gives the minimum pairwise 3-D distance per step.

Evidence and verdict. Every mission boundary writes one JSON record: each agent's truth, each live vehicle's observed position, altitude, clock and drift, every receiver's view of every task, and events including the fail-stop. The identical configuration is then flown on the point-mass backend for comparison. A report derives all metrics from the log alone and evaluates a test card mechanically (Table 2).

6 SITL results

Setup. ArduCopter quad SITL, `--speedup 5`, four instances, eight tasks, UAV 2 physically failed at $t = 4\,\text{s}$, fresh instances for every run. Runs were made on the team laptop (WSL2, ArduPilot 4.6.0-beta1) and in a 4-vCPU Linux container (ArduPilot 4.8.0-dev, built from source).

Table 2: Pass criteria of the multi-vehicle test card (LT3); every run must meet all.

P1	every task completed and the mission completed
P2	a physical failure, exactly one fail-stop sent to that vehicle, confirmed by the autopilot
P3	the held vehicle actually stopped (final speed below 0.5 m/s)
P4	the orphaned work was recovered by the survivors
P5	no concurrent ownership (zero task-seconds with two live owners)
P6	deconfliction held (no step below 5 m 3-D separation)
P7	lock-step stayed honest (every vehicle's drift below 1 s)
P8	the deterministic comparison run completed

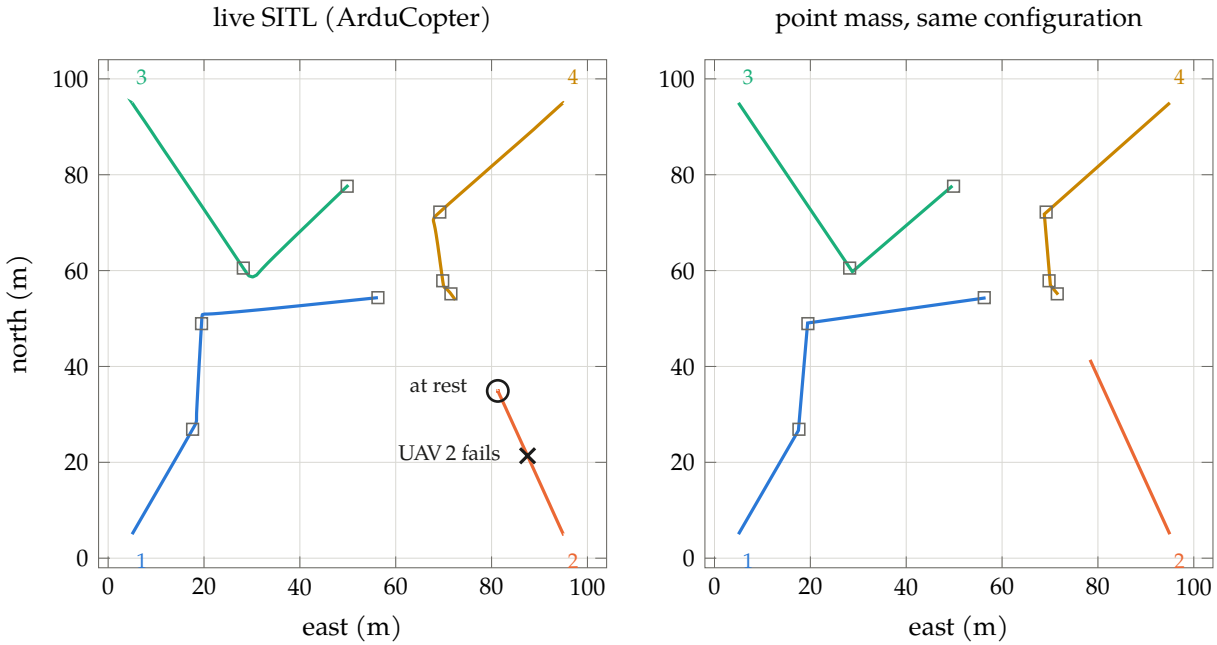


Figure 3: One recorded mission (run `c5`) seen from above. Left: tracks as each autopilot reported them, including UAV 2 braking after its fail-stop (× where it failed, ○ where it came to rest). Right: the point-mass run of the same configuration, whose failed agent stops dead. Squares are tasks; numbers mark each vehicle's start.

Results. Every container run met every criterion of Table 2. On the laptop, made before the separation metric (P6) existed, three runs completed 8/8 tasks with failure detection at 1.75 s, recovery after the failure in 6.75–7.25 s, maximum drift 0.45–0.50 s and zero concurrent ownership; the braked copter travelled 15.7–16.5 m before coming to rest. In the container, with stock vehicle parameters (Table 3, stock column), missions took 14.25–14.75 s against 8.50 s for the point-mass run of the same configuration, recovery took 7.00–7.25 s (2.50 s), and the minimum separation never fell below 13 m. A mixed swarm of three copters and one rover, with the rover failed at 8 s and held in `HOLD`, completed in 16.00 s with recovery in 2.25 s. Under a jammer, live detection took 5.00 s and recovery 7.25 s, with no concurrent ownership (the point-mass run of the same configuration showed 1.50 task-seconds: a different trajectory through the same protocol, not an error bar).

What only the firmware showed. A braked copter at cruise speed travels about 16 m before it is at rest: the safe hold is a stop command, not an instantaneous stop, whereas the simulated failed agent stops dead (Figure 3). The SITL binary exits when its parent process exits, so the launcher must keep a waiting parent per instance. With several vehicles a telemetry backlog is the normal case, and reading the head of the queue presents seconds-old positions as current; the backend drains to the newest report. And instances do not all run at the speedup requested: under CPU contention one instance's clock advanced about 17 % faster than another's, which anchoring records as growing drift rather than hiding.

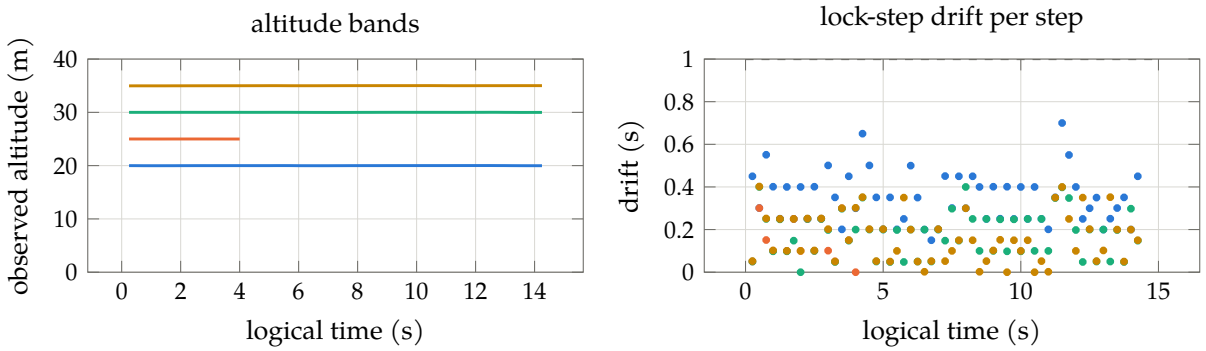


Figure 4: The same run: every copter held its own altitude band (UAV 2's band ends at its failure, after which it is held, not stepped), and the drift of each vehicle's clock ahead of logical time stayed bounded, well under the 1 s limit of criterion P7.

Table 3: Position-control tune: confirmation flights (top) and the LT3 mission (bottom).

	stock	tuned
settle time, 40 m step	11.05 s	6.10 s
overshoot	0.72 m	0.20 m
`BRAKE` from about 8 m/s	16.9 m	10.6 m
mission time (3 runs)	14.25–14.75 s	11.75–12.00 s
recovery after the failure	7.00–7.25 s	4.75–5.00 s
distance travelled after the fail-stop	15.6–17.4 m	12.9–13.8 m
minimum 3-D separation	13.0–14.9 m	16.4–16.6 m
live / point-mass mission time	1.71	1.40
test card	3/3 pass	3/3 pass

Changed: `PSC_NE_POS_P` 1 → 2, `PSC_NE_VEL_I` 1 → 1.5, `PSC_NE_JERK` 5 → 10, `WP_ACC` 2.5 → 5, `ATC_ANGLE_MAX` 30 → 35 (`config/sitl/copter-tuned.parm`).

7 Tuning the vehicles' position control

The flights exposed three things the copters' own control did poorly: the 16 m braking distance, legs about 1.7× slower than the point-mass model, and arrivals that overshot and crept back. All three are ArduCopter horizontal position control, so we tuned it on the simulated quad by measurement. Four SITL copters fly, in parallel, four 40 m GUIDED steps and one leg cut by `BRAKE` at 8 m/s for each candidate parameter set; trials are scored on settle time (to within 0.3 m, staying), overshoot, final error and braking distance; a candidate that rings, overshoots by more than 1.5 m, strays in altitude or rebounds after braking is disqualified outright. A staged coordinate search over safety-bounded values ran 104 trials; the winner was then flown 8 times against 8 flights of the stock parameters (Table 3, Figure 5). The inner attitude-rate loops were not touched. This is a tune of the simulated quad, not of any aircraft.

The search also mapped the boundaries. Doubling the position gain was the largest single gain; lowering the velocity gain or removing its derivative term made the vehicle ring; doubling the braking jerk again made it rebound by more than a metre after stopping. A tighter arrival tolerance did *not* shorten missions, because a task completes the first time a vehicle passes within the tolerance on the way in, so overshoot never delays completion; mission time is set by acceleration. The braking gain is smaller in the mission than in the fixed-speed test because the tuned copters are faster when they fail. The harness itself needed one correction: ArduCopter reads its leg acceleration limits only when GUIDED initialises, and re-selecting GUIDED while in it is a no-op, so early shaping trials flew with the previous candidate's limits; trials now pass through `BRAKE` first and that stage was re-flown.

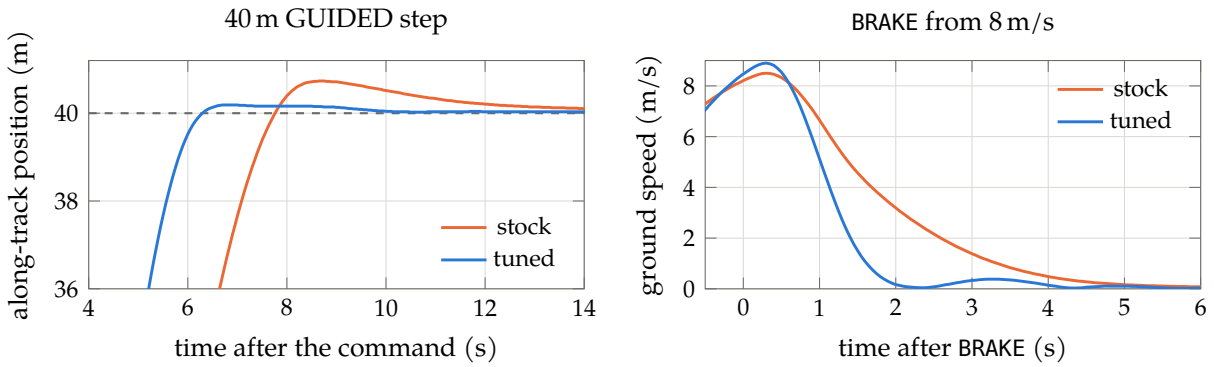


Figure 5: Step and braking response on the SITL quad, stock versus tuned (one recorded flight each). The stock controller overshoots and creeps back; the tuned one settles sooner. Under BRAKE, deceleration builds faster with the higher jerk limit.

8 Limitations and readiness

- **Not flight.** SITL runs real autopilot firmware on a simulated airframe. No physical flight has been performed, and the tuned parameters must never be flown on a real aircraft.
- **Modelled radio.** The links, the jammer and the failure are the simulator's models; the radio model is an analytical abstraction, not calibrated RF.
- **Non-reproducible live numbers.** SITL runs vary between runs and machines; they are reported as ranges and never as deterministic results.
- **Limited generality.** One main scenario family (four aircraft, a 100 m square, one jammer) plus a two-aircraft counterexample and a small fleet comparison.
- **Lock-step, not free-running.** Logical time is stepped against each vehicle's clock; a free-running integration remains future work.

Against the usual readiness scale [13], the coordination function is validated in simulation (TRL 3). The multi-vehicle SITL test (LT3) passed on two machines with a mechanical verdict; a TRL 4 claim for the coordination function additionally requires the protocol to run as separate processes over a real network that is partitioned and healed (LT5), which is not reported here.

9 Conclusion

Making every decision depend only on evidence a UAV actually received, holding ownership through leases that each replica ages on its own clock, and refusing to equate silence with failure gives a swarm that keeps completing its mission exactly where agreement is impossible. The same logic, unchanged, has driven four real autopilots through a mission in which one vehicle was physically stopped and its work was taken over by the others, and every container run passed a mechanical test card. What remains between this and a field claim is a real network, real radios and real airframes.

References

[1] European Union Aviation Safety Agency, *Safety information bulletin 2022-02r4: Global navigation satellite system outages and alterations leading to communication / navigation / surveillance degradation*, EASA, 2026.

[2] O. Osechas, F. Fohlmeister, T. Dautermann, and M. Felux, "Impact of GNSS-band radio interference on operational avionics," *NAVIGATION: Journal of the Institute of Navigation*, vol. 69, no. 2, 2022. DOI: 10.33012/navi.516.

[3] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *Journal of the ACM*, vol. 32, no. 2, pp. 374–382, 1985. DOI: 10.1145/3149.214121.

[4] T. D. Chandra and S. Toueg, "Unreliable failure detectors for reliable distributed systems," *Journal of the ACM*, vol. 43, no. 2, pp. 225–267, 1996. DOI: 10.1145/226643.226647.

[5] C. G. Gray and D. R. Cheriton, "Leases: An efficient fault-tolerant mechanism for distributed file cache consistency," in *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles (SOSP)*, 1989, pp. 202–210. DOI: 10.1145/74850.74870.

[6] ArduPilot Development Team, *ArduPilot: Open-source autopilot (ArduCopter, ArduRover, software-in-the-loop simulator)*, 2026.

[7] J. A. Tran *et al.*, "SwarmDAG: A partition tolerant distributed ledger protocol for swarm robotics," *Ledger*, vol. 4, 2019. DOI: 10.5195/ledger.2019.174.

[8] L. Lu *et al.*, "Communication-aware distributed task allocation for synchronous multi-UAV collaboration," *Aerospace Science and Technology*, p. 113 589, 2026. DOI: 10.1016/j.ast.2026.113589.

[9] S. G. Loizou and C. C. Constantinou, "Multi-robot coverage on dendritic topologies under communication constraints," in *55th IEEE Conference on Decision and Control (CDC)*, 2016, pp. 43–48. DOI: 10.1109/CDC.2016.7798244.

[10] S. Papaioannou *et al.*, "Distributed search planning in 3-D environments with a dynamically varying number of agents," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 2023. DOI: 10.1109/TSMC.2023.3240023.

[11] Y. K. Lopes *et al.*, "Supervisory control theory applied to swarm robotics," *Swarm Intelligence*, vol. 10, pp. 65–97, 2016. DOI: 10.1007/s11721-016-0119-0.

[12] M. Cao *et al.*, *Cooperative aerial robot inspection challenge: A benchmark for heterogeneous multi-UAV planning and lessons learned*, 2025. arXiv: 2501.06566.

[13] European Commission, *Horizon 2020 work programme 2014–2015, general annexes, G. technology readiness levels (TRL)*, 2014.

A Reproducing the results

```
# deterministic results (any machine, byte-identical)
eudis-swarm --scenario demo-quorum-floor
eudis-swarm --scenario baseline-quorum-floor
eudis-swarm-sweep --param communication-range --from 30 --to 130 --steps 11 \
  --seeds 20 --jammer 40,50,15,3,8 --task-control distributed --output d.csv

# multi-vehicle SITL test card (ArduPilot built: ./waf copter rover)
scripts/sitl-lt3.sh 3 5                                  # stock vehicles
COPTER_PARAMS=config/sitl/copter-tuned.parm scripts/sitl-lt3.sh 3 5
python scripts/sitl_swarm_report.py --check lt3-*/run*.jsonl

# the position-control tune
scripts/sitl-swarm.sh 4 5 && python scripts/sitl_tune.py --workers 4

# this paper: data from the committed logs, then XeLaTeX
python paper/tools/make_data.py && make -C paper
```

The evidence logs behind Sections 6 and 7 are committed under `docs/data/sitl/`, and `paper/tools/make_data.py` recomputes every number quoted from them.